

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Constraint-Based Problem Solving

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5– Naturalization (BL5,BL6)	Assessment:	Assessment Tool 1 – Constraint Based Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	<i>Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.</i>		
Student Name:	P . Sai Charan Tej	Reg. No.:	192465048
Section / Batch:		Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Apply backtracking techniques systematically for combinatorial problem solving.
- Clearly classify problems under P, NP, NP-Hard, and NP-Complete categories wherever required.
- Provide proper justification, state-space representation, and complexity analysis for all solutions.
- Neat presentation and logical explanation are mandatory.

Question Type	Focus Area	Questions
Constraint Based Problem Solving	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete.	Q1

SECTION A – CONSTRAINT-BASED PROBLEM SOLVING

- 1) Partition the vertices of a graph into cliques such that each vertex belongs to exactly one clique. Clearly define constraints related to clique formation. Analyze how constraints affect partition feasibility. Develop a backtracking-based solution. Apply pruning to reduce unnecessary partitions. Justify correctness and discuss NP-Hard classification.

Code of Conduct

I P . Sai Charan Tej(192465048)__(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%				
Constraint Analysis	25%				
Solution Development	25%				
Application of Concepts	15%				
Justification	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Algorithm

Step 1: Start with the given graph and initialize an empty list of cliques.

Step 2: Select the next unassigned vertex.

Step 3: Try to add the selected vertex to each existing clique.

Step 4: Before adding the vertex, check the clique constraint:

- The selected vertex must be adjacent to **every vertex already present** in that clique.
- If it is not adjacent to any member, it cannot be added to that clique.

Step 5: If the vertex cannot be added to any existing clique, create a **new clique** containing that vertex.

Step 6: Use **backtracking** to explore different possible clique assignments.

Step 7: Keep track of the partition having the minimum number of cliques found so far.

Step 8: Apply **pruning**:

- If the current number of cliques is already greater than or equal to the best solution found, stop exploring that branch.
- Avoid creating unnecessary duplicate partitions.

Step 9: Continue until all vertices are assigned to exactly one clique.

Step 10: The partition with the smallest number of cliques is the final optimal clique partition.

Correctness

The algorithm assigns every vertex exactly once. A vertex is added to a clique only when it is adjacent to all existing members, so every resulting group satisfies the clique condition. Backtracking explores alternative valid assignments, while pruning only removes branches that cannot improve the current best solution. Therefore, the algorithm finds a minimum clique partition.

Complexity

Clique Partitioning is an **NP-Hard** problem. The backtracking approach may examine an exponential number of possible partitions in the worst case. Pruning can significantly reduce the practical search space, but the worst-case time complexity remains exponential.

OUTPUT:

QUESTIONS

Q1

Clique Partitioning Problem
(NP-Hard Problem)

100 marks

1/1 answered

Q1 Clique Partitioning Problem (NP-Hard Problem)

100 marks

Partition graph vertices into cliques such that each vertex belongs to exactly one clique. Clearly define constraints related to clique formation. Analyze how constraints affect partition feasibility. Develop a backtracking-based solution. Apply pruning to reduce unnecessary partitions. Justify correctness and discuss NP-Hard classification.

Your Code

```
1 def can_add(vertex, clique, graph):
2     """Check whether vertex can be added to the clique."""
3     for u in clique:
4         if not graph[u][vertex]:
5             return False
6     return True
7 def backtrack(vertex, graph, cliques, best_partition, best_count):
8     n = len(graph)
9     if vertex == n:
10         if len(cliques) < best_count[0]:
11             best_count[0] = len(cliques)
```

Input

```
6 7
1 2
1 3
2 3
```

Output

```
Minimum number of cliques: 2
Clique Partition:
Clique 1 : 1 2 3
Clique 2 : 4 5 6
```